

NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING PLACEMENT EXAMINATION FOR CS1010/A/E/S C language version

July 2025

Time allowed: 120 minutes

INSTRUCTIONS TO CANDIDATES

1. This is a **CLOSED BOOK** assessment. The maximum possible score is 100.
2. You may want to use the calculator embedded in Exemplify. No physical calculator is allowed.
3. You may assume that all relevant libraries have been included.
4. Do **NOT** look at the questions until you are told to do so.

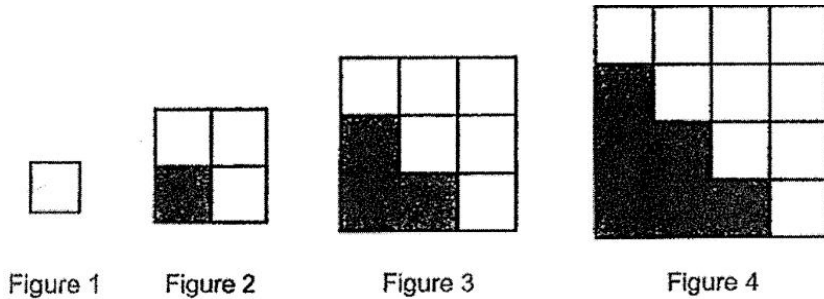
For examiners' use only							
Question	Q1	Q2	Q3	Q4	Q5	Q6	Total
Max	45	15	8	12	10	10	100
Score							

Q1. Multiple Choice Questions (MCQs)

[Total: 45 marks]

Each MCQ has one correct or best answer and is worth 3 marks.

1.1 The first four figures of a pattern are shown below. They each have 0, 1, 3 and 6 black squares, respectively. How many black squares are there in Figure 6?



- A. 8
- B. 10
- C. 12
- D. 15
- E. 21

1.2 Which of the following expressions is evaluated to **0**?

- A. $17\%5 == 2$
- B. $5/2 != 2.5$
- C. $1 != 1.0$
- D. $1 != '1'$
- E. None of the rest

1.3 What is printed by the following program?

```
int a[] = {4, 3, 2, 1, 0};
int b[] = {0, 0, 0, 0, 0};
int i;
for (i = 0; i < 5; i++) {
    b[i] = a[(i+2)%5];
}
printf("[");
for (i = 0; i < 4; i++) {
    printf("%d, ", b[i]);
}
printf("%d]\n", b[4]);
```

- A. [0, 1, 2, 3, 4]
- B. [1, 0, 4, 3, 2]
- C. [2, 1, 0, 4, 3]
- D. [4, 3, 2, 1, 0]
- E. None of the rest

1.4 Consider the following function `fool()`.

```
char * fool(int n) {
    if (n >= 50) {
        if (n > 60) {
            return "Ruler";
        } else if (n > 70) {
            return "Pencil";
        } else {
            return "Eraser";
        }
    } else if (n >= 40) {
        return "Clip";
    } else {
        return "Paper";
    }
}
```

Which of the following values will never be returned by the above function?

- A. **Ruler**
- B. **Pencil**
- C. **Eraser**
- D. **Clip**
- E. **Paper**

1.5 Which of the following statements best describes the function `do_thing(x, y)`, assuming that `x` and `y` are integers and `x > y > 0`?

```
int do_thing(int x, int y) {  
    int i = 1, m = x;  
    while (m%y != 0) {  
        i = i + 1;  
        m = i * x;  
    }  
    return m;  
}
```

- A. It returns `x` raised to the power of `y`, i.e., `xy`.
- B. It returns `y` raised to the power of `x`, i.e., `yx`.
- C. It returns the greatest common factor of `x` and `y`, i.e., the largest integer that fully divides both `x` and `y`.
- D. It returns the least common multiple of `x` and `y`, i.e., the smallest integer that is fully divided by both `x` and `y`.
- E. None of the rest

1.6 What does the following function return?

```
int unknown(int x, int y) {  
    if (x == 0) {  
        return y;  
    } else if (x < 0) {  
        return unknown(x+1, y+1);  
    } else {  
        return unknown(x-1, y-1);  
    }  
}
```

- A. It returns the value of `y + x`.
- B. It returns the value of `x * y`.
- C. It returns the value of `x - y`.
- D. It returns the value of `y - x`.
- E. None of the rest

1.7 What is printed by the following program?

```
int lst[4][4] = {{0, 0, 0, 0},
                 {0, 0, 0, 0},
                 {0, 0, 0, 0},
                 {0, 0, 0, 0}};

int k, m, n;
for (k = 0; k < 4; k++) {
    lst[k][0] = 1;
    lst[0][k] = 1;
}

for (m = 1; m < 4; m++) {
    for (n = 1; n < 4; n++) {
        lst[m][n] = lst[m-1][n-1] + lst[m][n-1];
    }
}

printf("[");
for (k = 0; k < 3; k++) {
    printf("%d, ", lst[3][k]);
}
printf("%d]\n", lst[3][3]);
```

- A. [1, 2, 4, 6]
- B. [1, 2, 4, 8]
- C. [1, 2, 4, 10]
- D. [1, 4, 6, 10]
- E. [1, 4, 10, 20]

- 1.8 Suppose there is a `random_real()` function that uniformly generates a random real number in the range $[0.0, 1.0]$, both ends inclusive (i.e., every number in the range will be generated with equal probability).

Consider the following program.

```
double total = 0;
int i, n = 1000000;
for (i = 0; i < n; i++) {
    total = total + 9 * random_real() + 3;
}
printf("%lf\n", total/n);
```

Which of the following is the *best estimated value* of the output?

- A. 4.5
- B. 6.0
- C. 7.5
- D. 9.0
- E. 12.0

1.9 What is printed by the following program?

```
#include <stdio.h>
int f(int *);

int main(void) {
    int x = -1;
    if ( f(&x) || x++ ) {
        x = 3;
    }
    printf("%d\n", x);
    return 0;
}

int f(int *x) {
    *x += 1;
    return *x + 1;
}
```

- A. -1
- B. 0
- C. 1
- D. 2
- E. 3

1.10 Suppose m and n are positive integers and $m < n$. Consider the following program.

```
int i, total = 0;
for (i = m; i < n; i++) {
    total++;
}
printf("%d\n", total);
```

Which of the following statements can be used to replace the code in the box and still produce the same output?

- A. `total += n - m - 1;`
- B. `total += n - m;`
- C. `total += n - m + 1;`
- D. `total += n*(n-1)/2;`
- E. None of the rest

1.11 Assuming that both x and k are positive integers, which of the following mathematics formulae equals to the return value of the function call `mystery(x, k, 0)`?

```
int mystery(int x, int k, int i) {
    if (k > i) {
        return 2 * mystery(x, k, i+1);
    }
    return x;
}
```

- A. k^x
- B. $2^k * x$
- C. $2^{k+1} * x$
- D. $2 * x^k$
- E. None of the rest

- 1.12 The volume of a balloon shrinks by 10% every day due to gas leakage. For example, suppose the balloon has initial volume 100. Its volume will become 90 and 81 on day 1 and day 2 respectively.

The function `get_volume(vol, n)` returns the volume of a balloon on day `n`, assuming `n` is a positive integer. The first parameter `vol` is the initial volume of a balloon.

For example, function call `get_volume(100, 1)` should return 90.0 and function call `get_volume(100, 2)` should return 81.0.

Which of the following is a correct implementation of `get_volume()` function?

(i)

```
double get_volume(int vol, int n) {  
    while (n >= 0) {  
        vol = vol * 0.9;  
        n = n - 1;  
    }  
    return vol;  
}
```

(ii)

```
double get_volume(int vol, int n) {  
    int i;  
    for (i = 0; i < n; i++) {  
        vol = vol * 0.9;  
    }  
    return vol;  
}
```

(iii)

```
double get_volume(int vol, int n) {  
    return vol * pow(0.9, n);  
}
```

- A. (i) only
- B. (ii) only
- C. (i) and (ii) only
- D. (ii) and (iii) only
- E. None of the others

1.13A **palindrome** is a word that can be read the same way in either direction. Examples of palindrome include:

- "I"
- "DID"
- "NOON"
- "ROTATOR"

The function `is_palindrome(s)` returns 1 if parameter `s` (less than 20 characters) is a palindrome, or 0 otherwise. You may assume that `s` is a string contains upper-case alphabet (i.e. upper-case English letters) only.

Which of the following is the correct implementation of the function `is_palindrome(s)`?

(i)

```
int is_palindrome(char *s) {
    char new_s[20] = "";
    int i, len = strlen(s);
    for (i = len-1; i >= 0; i--) {
        new_s[i] = s[len-i-1];
    }
    return strcmp(s, new_s) == 0;
}
```

(ii)

```
int is_palindrome(char *s) {
    int i, len = strlen(s);
    for (i = 0; i < len; i++) {
        if (s[i] != s[len-i-1]) {
            return 0;
        }
    }
    return 1;
}
```

(iii)

```
int is_palindrome(char *s) {
    int i, left = 0, right = strlen(s)-1;
    while (left < right) {
        if (s[left] != s[right]) {
            return 0;
        }
    }
}
```

```

        left++;
        right--;
    }
    return 1;
}

```

- A. (i) only
- B. (i) and (ii) only
- C. (ii) and (iii) only
- D. (i), (ii) and (iii)
- E. None of the rest

1.14 The following function **power_of_5(low, high)** accepts two positive integers **low** and **high** as parameters. It returns the number of integers in the range [**low**, **high**] (both ends inclusive) that are power of 5. You may assume that **high > low > 1**.

For example, there are 3 integers in the range [2, 200] that are power of 5. They are: 5 ($= 5^1$), 25 ($= 5^2$) and 125 ($= 5^3$). Therefore, function call **power_of_5(2, 200)** should return 3.

```

int power_of_5(int low, int high) {
    int n = 5, count = 0;
    while (n <= high) {
        XXXXXXXXXXXXXXXX // line 4 is hidden
        count++;
    }
    n = n * 5;
}
return count;
}

```

Line 4 of the above function is hidden from you (shown as **XXXXXXXXXXXXXX**). Which of the following can be substituted into that line to produce the desired result?

- A. **if (n > low) {**
- B. **if (n >= low) {**
- C. **while (n > low) {**
- D. **while (n >= low) {**
- E. None of the rest

- 1.15 Suppose we have an array of 20 points on a two-dimensional plane, where each point is represented by its integer x- and y-coordinates. The following program fragment sorts all the points in ascending order of their x-coordinates, and for those with the same x-coordinates, in ascending order of their y-coordinates.

```
typedef struct {
    int x, y;
} point_t;

// Return 1 if point p is less than point q,
// or 0 otherwise
int less_than(point_t p, point_t q) {
    return /** Part a */ ;
}

void sort(point_t points[20]) {
    int i, done = 0;
    while (!done) {
        done = 1;
        for (i = 0; i < 20; i++) {
            if ( /** Part b */ ) {
                point_t swap = points[i];
                points[i] = points[i+1];
                points[i+1] = swap;
                done = 0;
            }
        }
    }
}

int main() {
    // read 20 points from user input (omitted)
    sort(points);
    return 0;
}
```

Which of the following can be substituted into **/** Part a */** and **/** Part b */** to produce the desired result?

- A. Part a: `p.x < q.x && (p.x == q.x && p.y < q.y)`
 Part b: `less_than(points[i], points[i+1])`
- B. Part a: `p.x < q.x || (p.x == q.x && p.y < q.y)`
 Part b: `less_than(points[i], points[i+1])`

- C. Part a: `p.x < q.x || (p.x == q.x && p.y < q.y)`
 Part b: `! less_than(points[i], points[i+1])`
- D. Part a: `p.x > q.x || (p.x == q.x && p.y > q.y)`
 Part b: `! less_than(points[i], points[i+1])`
- E. None of the rest

Q2.

[Total: 15 marks]

Write down the output of the following program fragments. Each question is worth 3 marks.

2.1

```
char s1[7] = "ace", *s2 = "bdf";
strcat(s1, s2);
printf("%d\n", strlen(s1));
```

2.2

```
int num = 13579, val = 0, k = 1;
while (num > 0) {
    val += num%10*2%10*k;
    num /= 10;
    k *= 10;
}
printf("%d\n", val);
```

2.3

```
int lst [3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

int r, c, total = 0;
for (r = 0; r < 3; r++) {
    for (c= 0; c < 3; c++) {
        if ( (r+c)%2 != 0 ) {
            total += lst[r][c];
        }
    }
}

printf("%d\n", total);
```

2.4

```
#include <stdio.h>

void swap(int *, int *);

int main() {
    int a = 4, b = 7, *ptr;
    ptr = &b;
    swap(&a, ptr);
    printf("%d\n", a);
    return 0;
}

void swap(int *p, int *q) {
    *p += *q;
    q = p;
}
```

2.5

```
#include <stdio.h>

typedef struct {
    char code[10];
    int numStu;
} module_t;

void f(module_t);

int main() {
    module_t list[] = {"CS1010", 300}, {"CS1231", 100};
    f(list[1]);
    printf("%d\n", list[1].numStu);
    return 0;
}

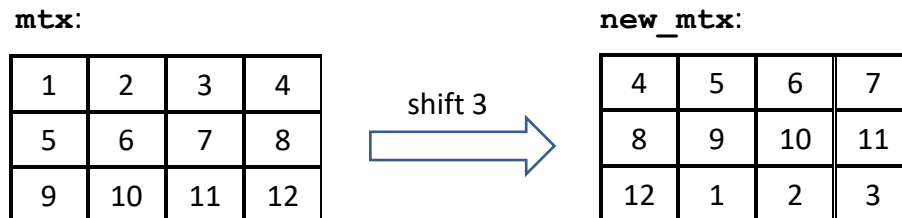
void f(module_t m) {
    --m.numStu;
}
```

Q3.

[8 marks]

Given a two-dimensional matrix **mtx** of size 3*4, we want to create another matrix **new_mtx** in which every element of **mtx** is shifted to the left by a shift amount.

For example, given the matrix **mtx** on the left below and a shift amount of 3, the matrix **new_mtx** on the right is created.



In the above example, elements in **mtx** are shifted to the left by 3 positions when appearing in **new_mtx**. If an element is shifted out of the matrix, it “re-enters” at the back of the matrix (see elements 1, 2 and 3 for examples). One way to do this is to “linearise” the element positions in **mtx** and map to their respective new positions in **new_mtx**. For example, **mtx[1][2]** is the 7th element in **mtx** and its new position (index) in **new_mtx** is 4 (= 7 - 3).

The following function **shift(mtx, shift_amt)** completes the above task. The parameter **shift_amt** is a positive integer indicating the shift amount of each element.

Fill in the missing part (which is a single line expression) in the dashed box.

```
void shift(int mtx[3][4], int shift_amt) {
    int r = 3, c = 4;    // number of rows and columns
    int row, col, idx;
    int new_mtx[3][4] = {
        {0, 0, 0, 0},
        {0, 0, 0, 0},
        {0, 0, 0, 0},
    };

    for (row = 0; row < r; row++) {
        for (col = 0; col < c; col++) {
            idx = ;

            new_mtx[idx/c][idx%c] = mtx[row][col];
        }
    }
    // the rest code omitted
}
```


Q4.

[Total: 12 marks]

The following lyric is extracted from the Christmas song “The Twelve Days of Christmas”:

**On the first day of Christmas,
My true love sent to me
A partridge in a pear tree.**

**On the second day of Christmas,
My true love sent to me
Two turtle doves,
And a partridge in a pear tree.**

**On the third day of Christmas,
My true love sent to me
Three French hens,
Two turtle doves,
And a partridge in a pear tree.**

(a) [6 marks] Going by this logic, write a recursive function

```
int present_on_day(int n)
```

that returns the number of presents you receive from your true love on day **n**.

For example, function call **present_on_day(1)** returns 1 (i.e., 1 present on the first day) and function call **present_on_day(3)** returns 6.

(b) [6 marks] Write a recursive function

```
int present_thru_days(int n)
```

that returns the total number of presents you receive from day 1 to day **n**.

For example, function call **present_thru_days(1)** returns 1 and function call **present_thru_days(3)** returns 10 (1 present on the first day + 3 presents on the second day + 6 presents on the third day).

You may call the **present_on_day()** function written in part (a).

Take note of the following instructions that apply to both part (a) and part (b):

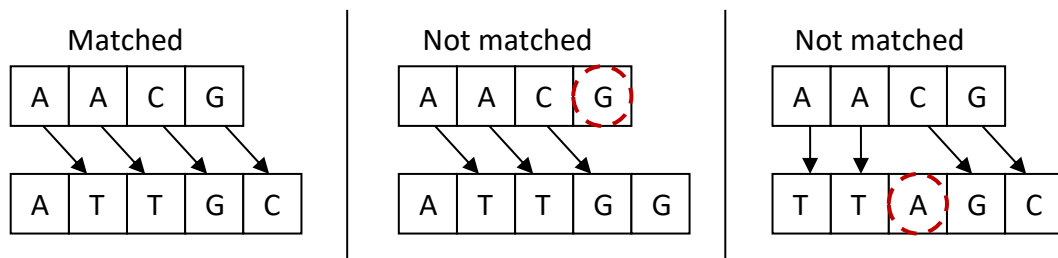
- You may assume that parameter **n** is a positive integer less than 13.
- You are to make recursive function calls instead of using math formula to calculate the number of presents. No loop is allowed either.
- You are not allowed to modify given function headers or create additional functions.

Q5.

[10 marks]

A DNA sequence is a string of four possible characters 'A', 'T', 'C' and 'G'. Among these four characters, 'A' and 'T' match each other and so do 'C' and 'G'.

Given two DNA sequences $s1$ and $s2$, $s1$ matches with $s2$ if all the characters in $s1$ can be matched with some characters in $s2$ **consecutively**. For example, the sequence "AACG" matches with the sequence "ATTGC" but not with "ATTGG" or "TTAGC", as explained below.



Write a function

```
int match(char *s1, char *s2)
```

that returns 1 if string $s1$ matches string $s2$, or 0 otherwise.

Take note of the following instructions:

- You may assume that given strings contain letters 'A', 'T', 'C' and 'G' only and are shorter than 20 characters.
- Partial mark may be awarded if your solution engages a nested loop.
- You are allowed to use the following String library functions: `strcmp()`, `strchr()`, `strstr()`, `strcpy()`, `strcat()` and `strlen()`.

Q6.

[10 marks]

A **subarray** is a continuous part (left-to-right) of a given array. For example, {5, -12, 9} and {6} are the subarrays of the array {5, -12, 9, 10, -4, 7, -8, 6} while {5, 9} and {6, -8, 7} are not.

The **subarray sum** is the sum of the values in the subarray. For example, subarray {5, -12, 9} has the sum 2.

Write a function

```
int max_subarray_sum(int a[8])
```

that takes an array of 8 integers as parameter and returns the maximum subarray sum among all the subarrays of **a**. To make it simple, you may assume that **a** contains at least 1 positive integer.

For instance, in the example array {5, -12, 9, 10, -4, 7, -8, 6}, the maximum subarray sum is 22 and the subarray that gives this sum is {9, 10, -4, 7}.

Take note of the following instructions:

- Full mark may be awarded if your solution engages a single loop only.
- Partial mark may be awarded if your solution engages a nested loop.
- You are not allowed to create additional array/string or functions.

=== END OF PAPER ===